

Reachability Rules: a Shared Algebra for Process-Graph Optimization, Signed-Link Leakage, and Cognitive-Agent Synthesis

Mathematical framework report

Csaba Hajdu

2026-06-14

Abstract

We isolate one algebraic object — a *reachability rule*: a parameter that selects which generators seed a monotone closure operator, evaluated against an admissibility predicate — and show it instantiates, with the same monotonicity and reduction theorems, in three otherwise-unrelated programs implemented in the HyMeKo framework: (A) Friedler-axiom process-network synthesis (P-graph optimization: MSG/SSG/ABB), (B) signed-link-prediction leakage auditing (neural networks), and (C) gatekeeper-bounded cognitive-agent synthesis (AKOIRE). The contribution is twofold: a small theory (§2) with proofs, and three instantiations (§3–§5) that each become a standalone article. We are explicit (§7) about what is a theorem, what is an empirically supported proposition, and what is a conjecture.

1 Introduction

Three problems in this codebase share a structure that is usually re-derived ad hoc in each. (A) PNS synthesis asks which products are *producible* from a set of operating units — a forward closure gated by the five Friedler axioms. (B) The signed-link leakage audit asks whether a held-out edge sign is *reachable* by a model’s message passing — a closure over an adjacency, the basis of the strict vs. transductive protocols. (C) The AKOIRE cognitive loop asks which refinements of an always-parseable *ambience* are *reachable* under a gatekeeper — a closure over accepted edits, bounded by a round cap. In all three a closure is computed, its seed/generator set is a free parameter, and an admissibility predicate filters the result. We name that parameter and study it once.

2 The theory

2.1 Closures and rules

Let $(L, \subseteq)$ be a finite lattice (in practice the powerset of a ground set of edges, materials, or refinements).

Definition 1 (Generator-seeded closure). A *closure family* is a monotone operator $C : 2^G \times L \rightarrow L$, where G is a set of *generators* and, for fixed generator set $\Gamma \subseteq G$, $C(\Gamma, \cdot)$ is a closure operator: extensive ($X \subseteq C(\Gamma, X)$), monotone ($X \subseteq Y \Rightarrow C(\Gamma, X) \subseteq C(\Gamma, Y)$), and idempotent. We additionally require monotonicity in the generators: $\Gamma \subseteq \Gamma' \Rightarrow C(\Gamma, X) \subseteq C(\Gamma', X)$.

Definition 2 (Reachability rule). A *reachability rule* R is a map assigning to a problem instance a generator set $\Gamma_R \subseteq G$. The *reachable set* under R from a base B is $\text{reach}_R(B) := C(\Gamma_R, B)$. Rules are ordered by $R \sqsubseteq R'$ iff $\Gamma_R \subseteq \Gamma_{R'}$ on every instance.

In every instantiation below the same three rules appear, forming a chain:

$$R_{\text{strict}} \sqsubseteq R_{\text{topo}} \sqsubseteq R_{\text{full}}. \quad (1)$$

R_{strict} admits only *confirmed* generators (training edges / established units / committed ambience); R_{topo} additionally admits *candidate connectivity* with labels withheld; R_{full} admits candidates with labels.

2.2 Core results

Theorem 1 (Monotonicity). For any base B , $R \sqsubseteq R' \Rightarrow \text{reach}_R(B) \subseteq \text{reach}_{R'}(B)$. In particular, along (1), $\text{reach}_{R_{\text{strict}}}(B) \subseteq \text{reach}_{R_{\text{topo}}}(B) \subseteq \text{reach}_{R_{\text{full}}}(B)$.

Proof. $R \sqsubseteq R'$ gives $\Gamma_R \subseteq \Gamma_{R'}$ by Definition 2; apply generator-monotonicity of C . $\square$

Theorem 2 (Reduction). Let C_0 be a domain's canonical closure with its conventional generator set Γ_0 . If $\Gamma_{R_{\text{strict}}} = \Gamma_0$, then $\text{reach}_{R_{\text{strict}}}(B) = C_0(B)$ for all B : the strict rule reproduces the pre-existing semantics exactly.

Proof. Immediate from $C(\Gamma_{R_{\text{strict}}}, B) = C(\Gamma_0, B) = C_0(B)$. $\square$

Theorem 2 is the safety guarantee: introducing the rule axis does not perturb a domain's established results — it is a conservative extension, checked by a regression test in each instantiation.

Theorem 3 (Bound admissibility). Let a branch-and-bound search prune a branch when a feasibility target $T \subseteq G$ is not contained in an optimistic reachable set $\text{reach}_R(B^\uparrow)$ (the closure under the maximal still-available generators). Widening R to $R' \sqsupseteq R$ keeps the bound admissible: no branch pruned under R' would have been retained under R , and the search remains exact.

Proof. By Theorem 1, $\text{reach}_R(B^\uparrow) \subseteq \text{reach}_{R'}(B^\uparrow)$, so $T \subseteq \text{reach}_R(B^\uparrow) \Rightarrow T \subseteq \text{reach}_{R'}(B^\uparrow)$: the wider rule prunes a (weak) superset of the branches the narrower one keeps feasible, never cutting an optimum. Exactness of the underlying B&B is preserved because the leaf feasibility test is unchanged. $\square$

3 Instantiation A — Process-graph optimization

Closure. G = operating units; $B = \Gamma_0$ = raw materials R ; C = the producibility fixpoint `close_producible`: admit a unit's outputs once all its inputs are reachable. This is exactly the closure the ABB reachability bound and SSG feasibility evaluate (`hymeko_pgraph`).

Rules. $\Gamma_{R_{\text{strict}}}$ = confirmed units (canonical PNS, so $\Gamma_{R_{\text{strict}}} = \Gamma_0$ and Theorem 2 applies); $R_{\text{topo}}/R_{\text{full}}$ additionally admit *candidate* units, differing only in whether candidate *costs* are counted — the cost layer, orthogonal to the closure.

Axioms as a second filter. The Friedler axioms S1–S5 are an admissibility predicate on the closed structure; A4/S4 (“every operating unit reaches a required product”) is itself a reachability statement, and the extension bundle (`axiom_extensions.rs`) shows orthogonal rules already sit *alongside* S1–S5 without contradiction. The reachability rule is the *reachability-axis* analogue: a refinement *regime* (the `regime.rs` Strategy seam) layered on MSG/SSG/ABB.

Verified (Rust, `hymeko_pgraph::reachability`): Theorem 2 as a unit test (R_{strict} equals `close_producible`); Theorem 1 as the monotone-closure test; a fixture where a product is producible *only* via a candidate unit (reachability strictly grows). Article A (§6).

4 Instantiation B — Signed-link leakage (neural networks)

Closure. G = edges (with optional signs); B = the node set; C = the message-passing reachability of a signed GNN/transformer — a node’s representation reaches the labels on edges in its receptive field.

Rules. $\Gamma_{R_{\text{strict}}}$ = training edges only; R_{topo} adds test-edge *topology* with signs masked to a neutral token; R_{full} adds test edges *with* signs (the convention these baselines self-report under). A train-label *shuffle* permutes training signs; the held-out AUROC under shuffle is the leakage probe.

Definition 3 (Leak). A method *leaks* under $(R, \text{shuffle})$ if its held-out AUROC under the shuffle stays above chance.

Proposition 1 (Necessity). *If the held-out label is not in $\text{reach}_R(B)$ then the method does not leak under $(R, \text{shuffle})$ (the readout has no access to it).*

Under R_{strict} the test sign is never reachable, so by Proposition 1 the strict-shuffle AUROC is chance. *Verified empirically:* 5-seed grid, 7 baselines $\times$ 5 datasets (350 runs); mean shuffled AUROC 0.515–0.545, $\sim$ 35 pp below real; epinions 0.500, slashdot 0.514 (clean to chance).

Proposition 2 (Reachability is not sufficient: the readout factor). *Label reachability under R is necessary but not sufficient for a leak. Leakage factors as*

$$\text{LEAK}(R, \text{model}) \iff [\text{label} \in \text{reach}_R(B)] \wedge [\text{readout is local}],$$

where a local readout exposes a held-out edge’s own label environment to its per-edge logit, and a diffuse (node-embedding) readout averages it away.

Empirical support. Under R_{full} (test signs reachable), an SGT with a node-embedding $\times$ MLP readout still drops to chance under shuffle (0.514, bitcoin-alpha, 60 ep) — reachable but not exploited; whereas a cycle-/path-feature readout leaks at 0.73 (2026-06-11). The factorization is the proposed headline of Article B; the decisive test is the R_{topo} sweep across readout families (wired and unit-tested; sweep pending).

Remark 1 (A residual the label-shuffle misses). On reddit-body the strict-shuffle AUROC is a *stable* 0.564 ± 0.024 , not chance. Because the shuffle permutes signs but preserves topology, a node-level sign prior carried by degree/position survives it. Formally this is leakage through a *generator the rule does not vary* (topology); isolating it needs a degree-preserving rewire control. (Conjecture 1.)

5 Instantiation C — Cognitive-agent synthesis (AKOIRE)

Closure. G = candidate refinements the agent may propose; B = the current *ambience* (an always-parseable source with a monotonic generation counter); C = the set of ambiances reachable by sequences of *gatekeeper-accepted* refinements. Each accepted refinement increments the generation; the closure is approached one step per loop round, bounded by `max_rounds`.

Rule & admissibility. The gatekeeper (the HyMeKo Layer-1 parser) is the admissibility predicate: a refinement is admitted iff the resulting source parses (the cognitive-layer analogue of the

Friedler axioms). The reachability rule selects which refinements are candidates — a *strict* agent reaches only refinements over confirmed structure; a *transductive* agent may reach over proposed-but-unconfirmed structure, with or without committing its semantics (topology vs. full), exactly mirroring (1).

Proposition 3 (Termination). *Since the generation counter is strictly monotone on acceptance and the loop is capped, the cognitive closure is approached by a finite, non-decreasing chain $B = A_0 \sqsubseteq A_1 \sqsubseteq \dots$ terminating in CONVERGED, EXHAUSTED, or CAPREACHED.*

Realized in `akoire` (the `CognitiveLoop` driver, `ScriptedSynthesizer` for determinism); the LLM-backed synthesizer is the production binding. Article C frames the agent as *reachable-refinement synthesis under a structural gatekeeper* — “structurally accountable AI”: the agent’s moves are exactly the admissible elements of a reachability closure.

6 The three articles

Program	Contribution	Status / evidence
A. P-graph opt.	Reachability <i>regimes</i> for PNS: the rule lattice over MSG/SSG/ABB; Thm. 2/3; candidate-unit synthesis.	<code>reachability.rs</code> + 4 tests; Pimentel fixtures (book_validation).
B. Neural nets	Leakage = reachability rule $\times$ readout locality (Prop. 2); the strict protocol; Gömb-strict survives.	350-run 5-seed grid; figure; R_{topo} wired + tested; sweep pending.
C. Cognitive agents	Structurally-accountable synthesis: agent moves = admissible reachable refinements (Prop. 3).	<code>akoire</code> crate + loop tests; LLM binding sketched.
0. Umbrella	This framework (§2): one algebra, three domains.	this report.

A detailed sequencing/venue plan is in `docs/plans/2026-06-14-research-program-reachability/`.

7 Epistemic status (honesty)

- **Theorems 1–3** are proved above and hold in every instantiation; 2 is additionally a passing regression test in domains A and B.
- **Propositions 1–3** are correct given their definitions; 2 (the readout factor) is *empirically supported*, not proved — it rests on the SGT-vs-cycle contrast and awaits the full R_{topo} sweep.
- The **cognitive-agent** instantiation (§5) is the least formally developed: the closure/gatekeeper picture is faithful to the `akoire` implementation, but “reachable-refinement synthesis” is a framing, not yet a theorem about agent competence.

Conjecture 1 (Topology-carried residual). On graphs where edge sign correlates with node topological position (e.g. reddit-body), held-out AUROC under $(R_{\text{strict}}, \text{shuffle})$ exceeds chance by a margin equal to the predictivity of a degree/position-only node prior; a degree-preserving rewire of the test topology removes it.